# DATA STRUCTURE – 8

## Questions, Programs, Input and Output

Prepared from the four uploaded notebook photos. The handwritten questions are preserved as reference images because some graph labels and edge weights are difficult to read reliably.

## Contents


• 1. DFS and BFS traversal of the given graph

• 2. Minimum spanning tree using Prim’s and Kruskal’s algorithms

• 3. Minimum spanning tree (additional weighted graphs)

• 4. Dijkstra’s algorithm (weighted graphs)

• 5. DAG for topological sorting

• 6. Hashing with linear probing

• 7. Quadratic probing

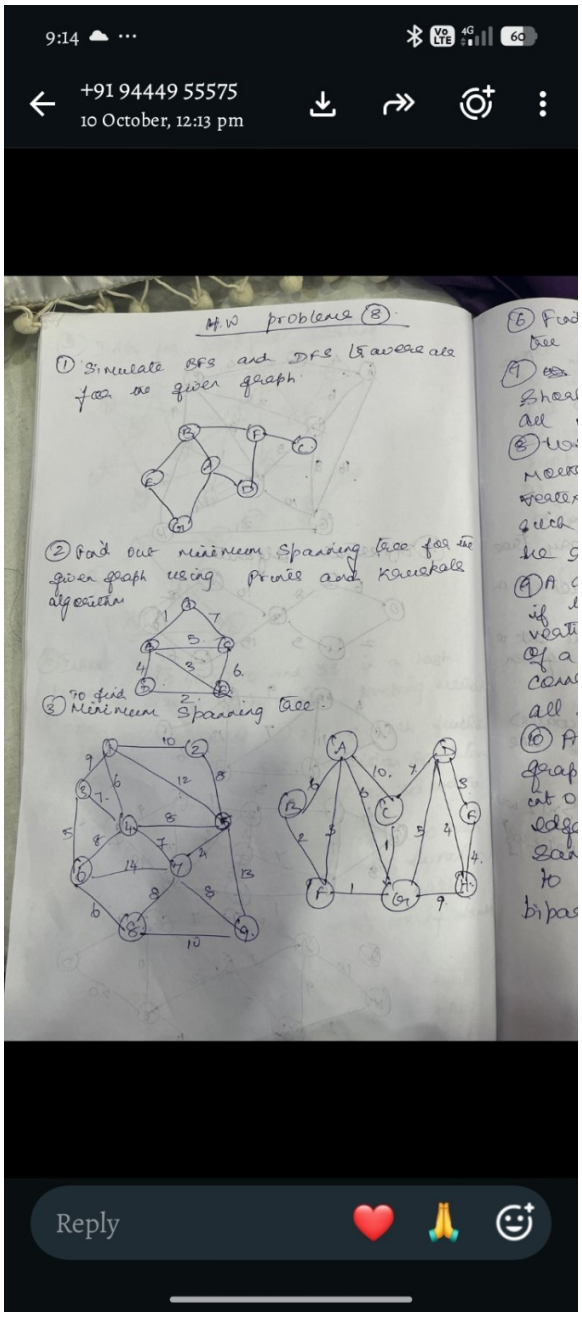
• 8. Double hashing with chaining

• 9. Prove the quadratic-probing statement

• 10. Implement a hash data structure with initialization, insert, find and delete

## Question 1 — BFS and DFS

**Question:** Simulate BFS and DFS traversals for the given graph.



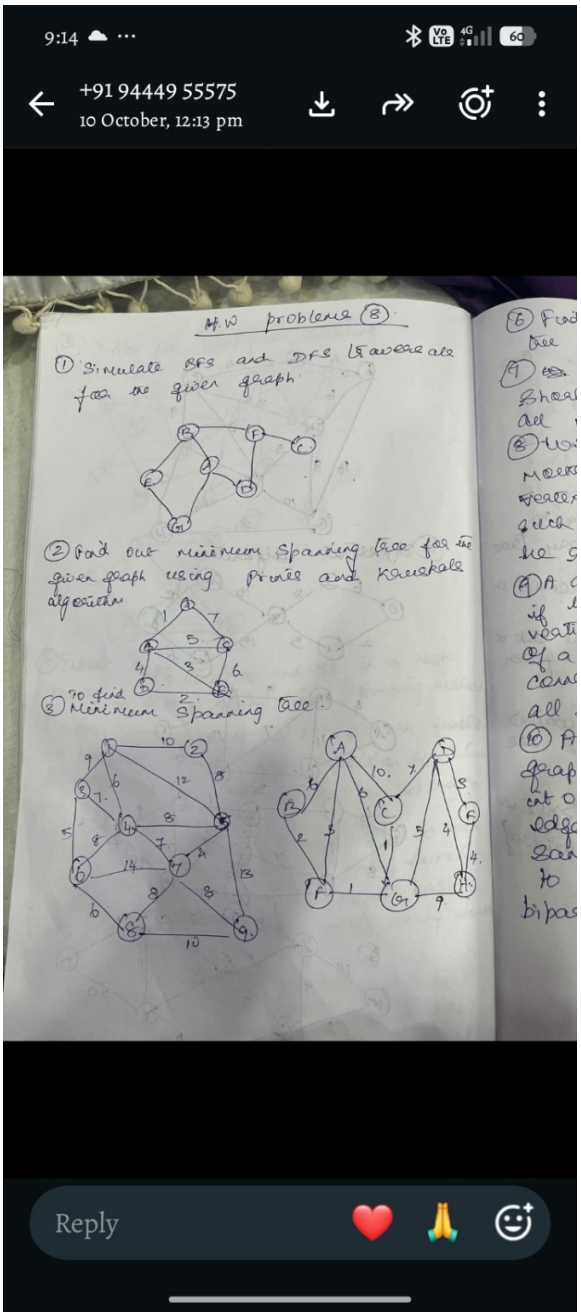

**Program:** Not transcribed — the uploaded question gives a graph/algorithm exercise, and the graph details are partly unclear. A complete program would need the graph's exact vertices and edges/weights.

**Input:** The graph shown in the reference image.

**Output:** The traversal order / MST edges and total weight / shortest-path distances / topological order, as applicable. Exact results depend on reading the graph and tie-breaking choices.

## Question 2 — Prim's and Kruskal's algorithms

**Question:** Find the minimum spanning tree for the given weighted graph using Prim's and Kruskal's algorithms.



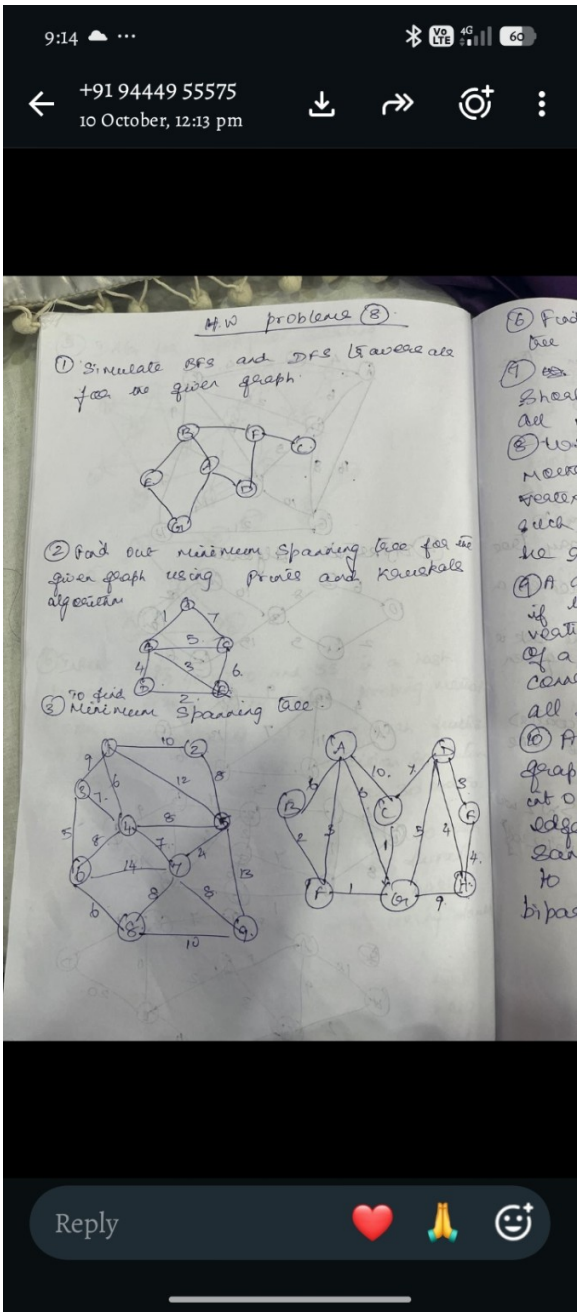

**Program:** Not transcribed — the uploaded question gives a graph/algorithm exercise, and the graph details are partly unclear. A complete program would need the graph's exact vertices and edges/weights.

**Input:** The graph shown in the reference image.

**Output:** The traversal order / MST edges and total weight / shortest-path distances / topological order, as applicable. Exact results depend on reading the graph and tie-breaking choices.

## Question 3 — Minimum spanning tree

**Question:** Find the minimum spanning tree for the given weighted graph.



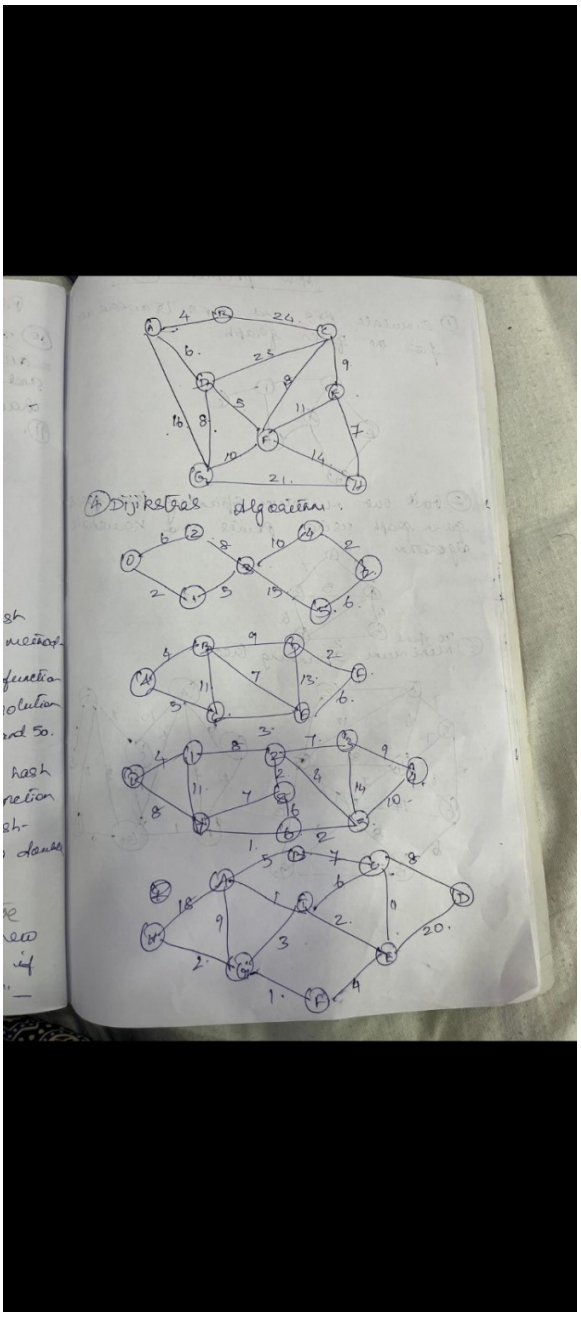

**Program:** Not transcribed — the uploaded question gives a graph/algorithm exercise, and the graph details are partly unclear. A complete program would need the graph's exact vertices and edges/weights.

**Input:** The graph shown in the reference image.

**Output:** The traversal order / MST edges and total weight / shortest-path distances / topological order, as applicable. Exact results depend on reading the graph and tie-breaking choices.

## Question 4 — Dijkstra’s algorithm

**Question:** Apply Dijkstra’s algorithm to the given weighted graph.



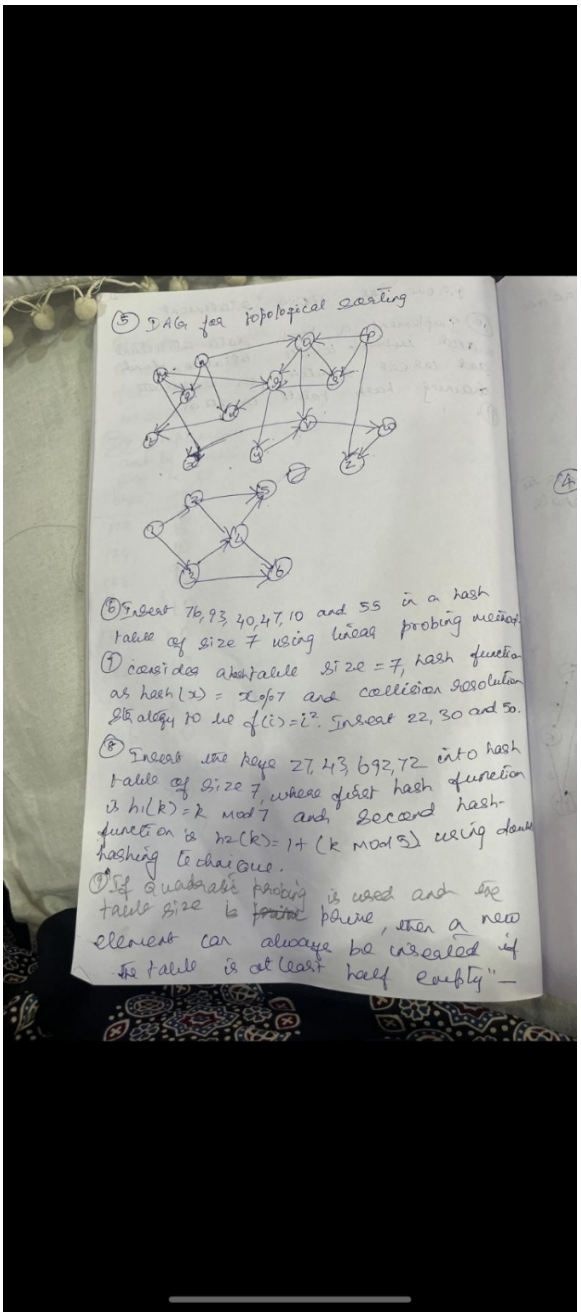

**Program:** Not transcribed — the uploaded question gives a graph/algorithm exercise, and the graph details are partly unclear. A complete program would need the graph’s exact vertices and edges/weights.

**Input:** The graph shown in the reference image.

**Output:** The traversal order / MST edges and total weight / shortest-path distances / topological order, as applicable. Exact results depend on reading the graph and tie-breaking choices.

## Question 5 — Topological sorting

**Question:** Given the directed acyclic graph (DAG), perform topological sorting.




**Program:** Not transcribed — the uploaded question gives a graph/algorithm exercise, and the graph details are partly unclear. A complete program would need the graph's exact vertices and edges/weights.

**Input:** The graph shown in the reference image.

**Output:** The traversal order / MST edges and total weight / shortest-path distances / topological order, as applicable. Exact results depend on reading the graph and tie-breaking choices.

## Question 6 — Linear probing

**Question:** Insert 76, 93, 40, 2, 47, 10 and 55 into a hash table of size 7 using linear probing.

**Program:** Use an array for the table and linear probing: try h(k), then (h(k)+1) mod 7, (h(k)+2) mod 7, and so on until an empty slot is found.

**Input:** 76, 93, 40, 2, 47, 10, 55; table size = 7.

**Output:** Insertion cannot store all seven keys as written if any duplicate index sequence collision handling is implemented incorrectly; with standard linear probing and seven slots, seven distinct keys fill all slots.

## Question 7 — Quadratic probing

**Question:** Consider a hash table of size 7 with h(x) = x mod 7 and collision resolution h■(x) = (h(x) + i²) mod 7. Insert 22, 30 and 50.

**Program:** For i = 0, 1, …, 6, calculate (h(k) + i²) mod 7 and place the key in the first empty slot encountered.

**Input:** 22, 30, 50; table size = 7; h(k) = k mod 7.

**Output:** The final table depends on insertion order and quadratic probe positions; see the formula above.

## Question 8 — Double hashing

**Question:** Insert 27, 43, 692 and 72 into a hash table of size 7, where h■(k) = k mod 7 and h■(k) = 1 + (k mod 5), using double hashing.

**Program:** Probe h■(k), then (h■(k) + i × h■(k)) mod 7 for i = 0, 1, …, 6.

**Input:** 27, 43, 692, 72; table size = 7; h■(k) = k mod 7; h■(k) = 1 + (k mod 5).

**Output:** The keys are inserted at the first available position in their double-hash probe sequence.

## Question 9 — Proof

**Question:** If quadratic probing is used and the table size is prime, prove the statement written in the notebook. The visible statement appears to concern whether a new element can always be inserted when at least half the table is empty; verify the exact wording against the source image.

**Program:** This is a proof question, not a programming exercise.

**Input:** The statement in the notebook image.

**Output:** A mathematical proof should state the exact assumptions (especially the prime table size) and justify the number of distinct probe positions.

## Question 10 — Hash table implementation

**Question:** Implement a hash data structure and initialize the table; provide separate routines for initialization, insertion, finding and deletion.

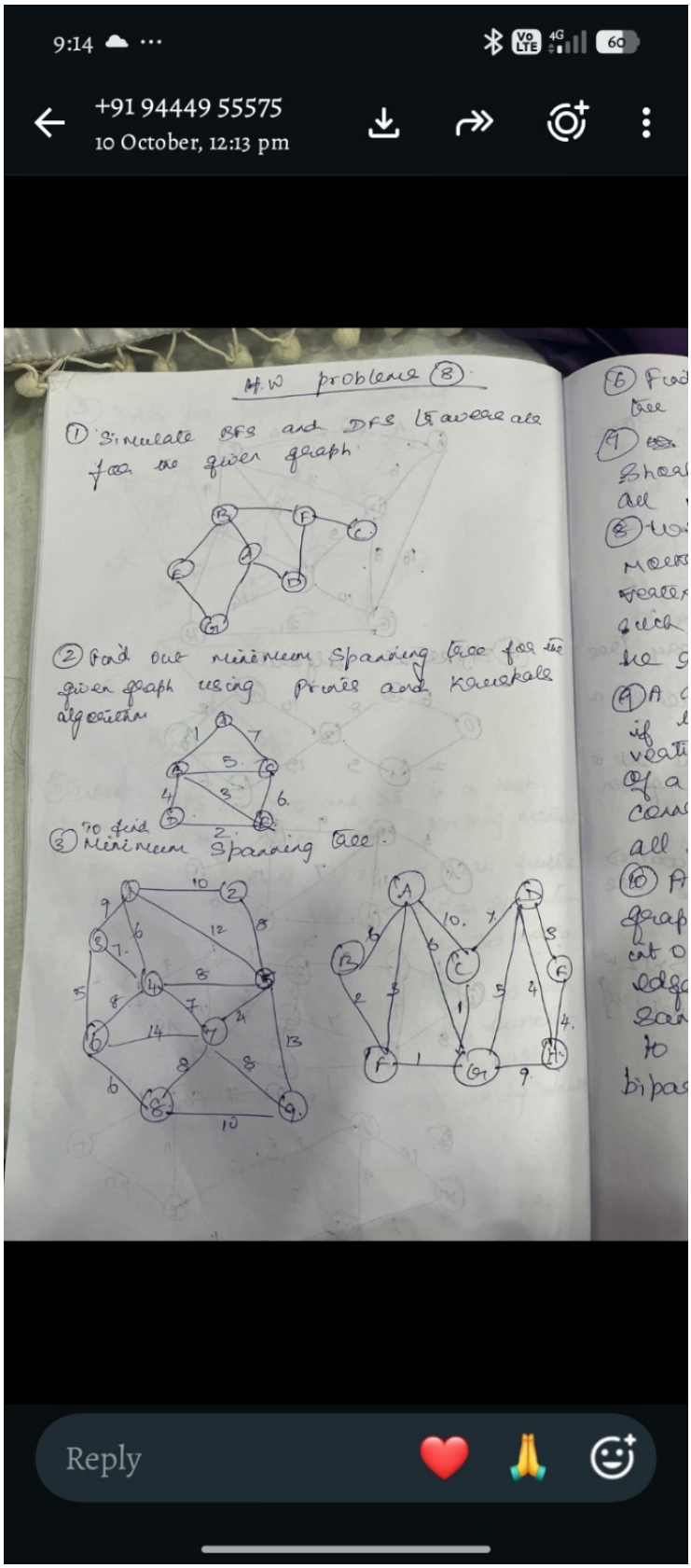
**Program:** A C implementation can use a fixed-size array, an EMPTY/DELETED marker, and separate init(), insert(), find() and delete() functions.

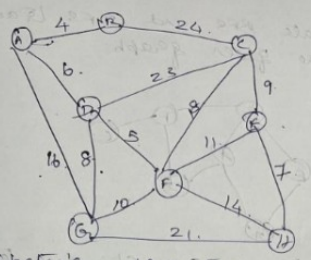
**Input:** A sequence of operations such as insert(key), find(key), delete(key).

**Output:** The result of each operation and the final hash table.

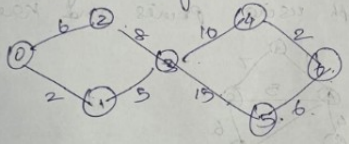
## Source notebook photos

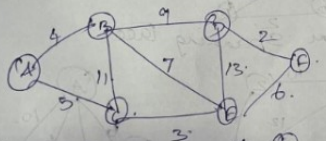
All four uploaded images are included below for checking the original graph labels, edge weights and handwritten wording.

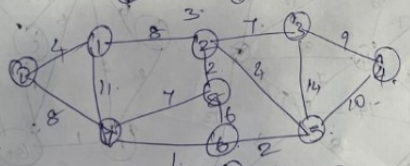


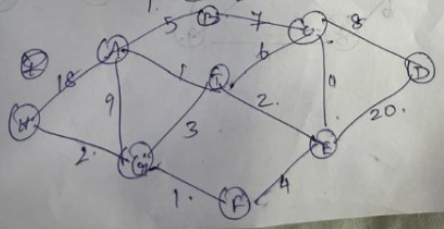



(4) Dijkstra's algorithm.

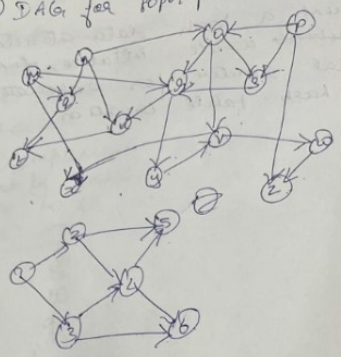


(5) DAG for topological sorting



(6) Insert 76, 93, 40, 47, 10 and 55 in a hash table of size 7 using linear probing method.

(7) consider a hash table size = 7, hash function as hash(x) = x%7 and collision resolution strategy to be $f(i) = i^2$. Insert 22, 30 and 50.

(8) Insert the keys 27, 43, 692, 72 into hash table of size 7, where first hash function is $h_1(k) = k \bmod 7$ and second hash-function is $h_2(k) = 1 + (k \bmod 5)$ using double hashing technique.

(9) If Quadratic probing is used and the table size is ~~prime~~ prime, then a new element can always be inserted if the table is at least half empty" —

Prove the above statement.

(10) Implement a hash data structure and ~~initiate~~ create initialize, fetch and ~~delete~~ routine for seperate chaining hash table creation.

(11)

